



FACULTAD DE INGENIERÍA

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 1

Los Algoritmos Greedy son juegos de niños



16 de septiembre de 2026

Valentina Romero
112724

Nicolas Araujo
114312

Ursino Ian Mika
104509

1. Algoritmo para ganar el juego

En este trabajo realizaremos un análisis del problema y propondremos un algoritmo greedy que obtenga la solución óptima al mismo.

1.1. Problema

Dos hermanitos, Sophia y Mateo, juegan un juego de monedas: dada una fila de n monedas de diferentes valores, en cada turno un jugador debe elegir alguna. Pero solo puede elegir la primera de la fila o la última. Al elegirla, la remueve de la fila y le toca al otro jugador, quien debe elegir otra moneda siguiendo la misma regla. Siguen agarrando monedas hasta que no quede ninguna. Quien gane será quien tenga las monedas que sumen el mayor valor.

Como Mateo es pequeño para entender el juego, Sophia elige por él. Sin embargo, Sophia es muy competitiva y quiere ganar siempre. Por lo que el problema consiste en que, dado n valores de monedas, debemos indicar qué monedas debe ir eligiendo Sophia para sí misma y para Mateo, de tal forma que se asegure de ganar siempre (Sophia siempre empieza jugando para sí misma).

1.2. Algoritmo Greedy

De forma intuitiva se ve que el algoritmo que resuelve nuestro problema es: Sophia debe elegir para sí misma la opción de mayor valor y para su hermano, la opción de menor valor.

```
1 def solucion(monedas):
2     ini = 0
3     fin = len(monedas) - 1
4     mon_s = []
5     mon_m = []
6     turno = 0
7
8     while ini <= fin:
9         izq = monedas[ini]
10        der = monedas[fin]
11        if turno % 2 == 0:
12            if izq > der:
13                mon_s.append(izq)
14                ini += 1
15            else:
16                mon_s.append(der)
17                fin -= 1
18        else:
19            if izq < der:
20                mon_m.append(izq)
21                ini += 1
22            else:
23                mon_m.append(der)
24                fin -= 1
25        turno += 1
26    return mon_s, mon_m
```

1.2.1. Complejidad del algoritmo

La complejidad del algoritmo propuesto para resolver el problema es $\mathcal{O}(n)$, siendo n la cantidad de monedas al inicio del juego. Esto se debe a que en cada iteración se elimina exactamente una moneda (Sophia elige una moneda para ella o para su hermano), por lo que el ciclo se ejecuta exactamente n veces. En cada iteración se realiza una cantidad constante de operaciones $\mathcal{O}(1)$, independientemente de los valores de las monedas. Por lo tanto, la variabilidad de los valores no afecta la complejidad temporal del algoritmo.

1.2.2. Variabilidad de los valores y optimalidad

La magnitud de las diferencias entre los valores de las monedas no afecta la optimalidad del algoritmo, siempre que se mantengan las condiciones bajo las cuales se garantiza la victoria. El objetivo considerado no es maximizar la ventaja de Sophia sobre Mateo, sino únicamente conseguir que su suma final sea mayor. Por lo tanto, es indistinto que Sophia gane por 5 o por 500 puntos.

En cada turno de Sophia, el algoritmo selecciona el extremo de mayor valor. El otro extremo continúa disponible para el turno siguiente de Mateo, en el cual se elige el menor de los dos extremos disponibles. En consecuencia, la moneda asignada a Mateo es menor o igual que la obtenida previamente por Sophia.

Sin embargo, la existencia de valores repetidos sí puede afectar el resultado: las comparaciones pueden dejar de ser estrictas y producir un empate. Por ello, la variabilidad no modifica la regla ni la complejidad del algoritmo, pero la distribución concreta de los valores puede determinar si la ventaja final es estricta. Si todos los valores son distintos y positivos, Sophia obtiene una ventaja estricta; si se permiten repeticiones, el algoritmo puede garantizar solamente que no pierda en ciertos casos.

1.2.3. ¿El algoritmo es greedy?

Sí, el algoritmo es greedy. De las monedas que son opción (estado actual), si Sophia elige para sí misma, toma la moneda que tiene mayor valor y si elige para su hermano, toma la moneda que tiene menor valor (regla greedy). De esta forma, maximizamos la suma acumulada de Sophia y minimizamos la suma acumulada de Mateo en sus respectivos turnos (óptimo local).

1.2.4. ¿Es óptimo?

Sí, lo demostraremos en la siguiente sección.

1.3. Demostración

Vamos a demostrar que el algoritmo greedy propuesto es óptimo (desestimando el caso de una cantidad par de monedas de mismo valor, ya que siempre terminaría en empate sin importar la estrategia de Sophia). Para esto demostramos por inducción que, dadas n monedas en donde Sophia empieza, si Sophia elige para sí misma la moneda de mayor valor y para su hermano la de menor valor, ella gana el juego. La proposición que vamos a demostrar en el paso inductivo será $P(h) \rightarrow P(h+2)$. Por lo que necesitaremos 2 casos base para poder cubrir tanto una cantidad par de monedas como una impar.

1. Casos base:

$$n = 1 \rightarrow [m_1]$$

Sophia empieza y elige la única moneda que hay. Por consiguiente, la suma acumulada de Sophia es mayor a la de Mateo, pues la de él queda en cero, y Sophia gana el juego.

$$n = 2 \rightarrow [m_1, m_2]$$

Sophia empieza y elige la moneda de mayor valor. Luego, Mateo se queda con la que sobra. Por consiguiente, la suma acumulada de Sophia es mayor a la de Mateo y gana el juego.

2. Paso inductivo por método directo:

Asumimos que $P(h)$ se cumple. Esto quiere decir que en un arreglo de h monedas donde Sophia empieza jugando, Sophia gana.

Si tomamos el caso base de $n = 1$, sin importar cuantas veces le sumemos 2, el arreglo va a terminar teniendo una cantidad impar de monedas. Podemos observar que le tocará jugar a Sophia cuando haya una cantidad impar de monedas disponibles. En cambio, si tomamos el caso base de $n = 2$, sin importar cuantas veces le sumemos 2, el arreglo va a terminar teniendo una cantidad par de monedas. En este caso, le tocará jugar a Sophia cuando haya una cantidad par de monedas disponibles.

En ambos casos podemos dividir el arreglo de $h + 2$ monedas en dos subarreglos de la siguiente forma:

$$[m_1, m_2, \dots, m_h, m_{h+1}, m_{h+2}] \longrightarrow [m_1, m_2, \dots, m_h] + [m_{h+1}, m_{h+2}]$$

El subarreglo de dos monedas representa los dos primeros turnos del juego. Como Sophia empieza jugando, ella se queda con la moneda de mayor valor y Mateo se queda con la moneda sobrante. Se puede ver que en este subarreglo gana Sophia. Sin importar si el arreglo tiene un tamaño impar o par, al sacarle dos monedas, la paridad se mantiene y, como vimos anteriormente, vuelve a ser el turno de Sophia. El hecho de que le toque a Sophia jugar en el arreglo de h monedas coincide con la hipótesis que asumimos verdadera y por ende, Sophia gana en este subarreglo. Que Sophia gane en ambos subarreglos significa que en ambos obtuvo una suma acumulada mayor que la de su hermano y como se cumple que:

$$a \geq b, c \geq d \longrightarrow a + c \geq b + d$$

Sophia gana el juego completo.

1.4. Mediciones

Para analizar el comportamiento del algoritmo, medimos el tiempo de ejecución para distintos tamaños de entrada. Luego, con los tiempos obtenidos, realizamos un ajuste por cuadrados mínimos utilizando una función lineal de la forma:

$$T(n) = c_1 n + c_2$$

Elegimos este modelo porque, según el análisis teórico realizado anteriormente, el algoritmo tiene complejidad temporal $O(n)$. Por lo tanto, se espera que el tiempo de ejecución aumente aproximadamente de forma lineal con el tamaño de la entrada.

1.4.1. Generación de los conjuntos de datos

Para realizar las mediciones se generaron conjuntos de datos de distintos tamaños. Cada conjunto representa una fila de monedas, cuyos valores son enteros generados aleatoriamente entre 1 y 1000 inclusive. Los valores se guardaron en el mismo orden en que fueron generados y separados por “;”.

Los tamaños utilizados fueron:

20, 25, 50, 100, 500, 1000, 5000, 10000, 20000, 50000,
100000, 200000, 500000, 1000000, 2000000, 5000000, 7500000, 10000000.

1.4.2. Metodología de medición

Para cada tamaño de entrada se realizaron cinco ejecuciones del algoritmo y se registró el tiempo de cada una. Luego, se calculó el promedio:

$$T(n) = \frac{1}{5} \sum_{i=1}^5 T_i(n)$$

De esta forma, se utilizó el promedio como el tiempo representativo para cada tamaño de entrada, reduciendo el efecto de posibles variaciones entre ejecuciones.

A partir de los tiempos obtenidos se realizó un ajuste por cuadrados mínimos utilizando la función:

$$\hat{T}(n) = c_1 n + c_2$$

Los valores de c_1 y c_2 se obtuvieron minimizando el error cuadrático entre los tiempos medidos y los tiempos dados por el ajuste.

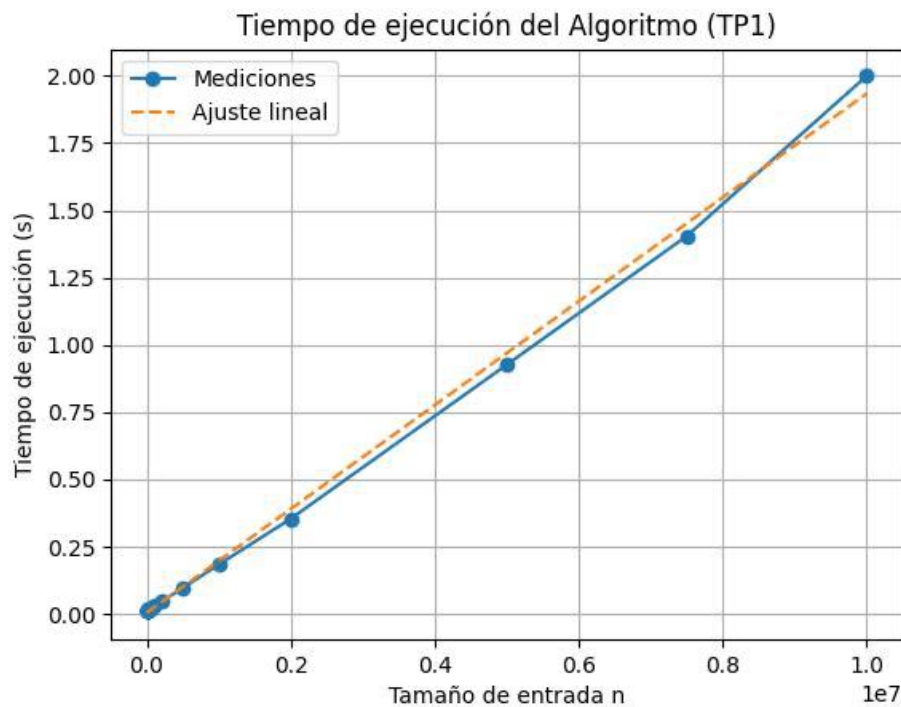


Figura 1: Tiempo de ejecución en función del tamaño de entrada y ajuste lineal.

En la Figura 1 se puede observar que los tiempos presentan una tendencia aproximadamente lineal a medida que aumenta el tamaño de la entrada. Esto coincide con la complejidad $O(n)$ obtenida en el análisis teórico.

1.4.3. Error absoluto

Para observar qué tan bien se ajusta la función lineal a las mediciones, se calculó el error absoluto para cada tamaño de entrada:

$$E(n) = |T(n) - \hat{T}(n)|$$

donde $T(n)$ es el tiempo medido y $\hat{T}(n)$ es el tiempo estimado por el ajuste lineal.

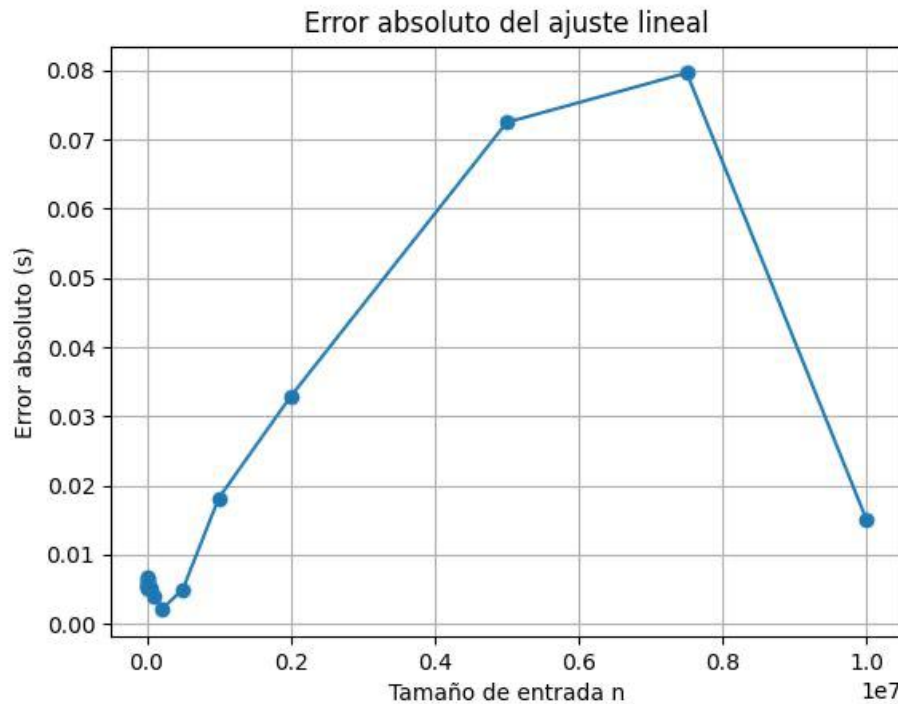


Figura 2: Error absoluto del ajuste lineal para cada tamaño de entrada.

La Figura 2 muestra la diferencia entre los tiempos medidos y los estimados por el modelo. El error puede variar debido a factores externos a la ejecución del algoritmo, como la carga del sistema o el tiempo necesario para leer los archivos de entrada.

1.5. Conclusiones

En este trabajo se ha analizado el juego de las monedas, donde dos jugadores (Sophia y Mateo) eligen monedas de una fila según ciertas reglas y Sophia, aplicando un enfoque greedy, busca ganar siempre. A través de la implementación del algoritmo propuesto, hemos logrado simular el comportamiento de los jugadores demostrando que todas las situaciones¹ desembocan en el triunfo de Sophia. Esto lo logramos aplicando iterativamente una simple regla: dadas las monedas disponibles en sus respectivos turnos, Sophia elige la moneda de mayor valor y Mateo la de menor valor.

La solución propuesta cumple con la función de optimizar la selección de monedas para Sophia, demostrando como los algoritmos greedy pueden ser efectivos en este tipo de situaciones. Pero, a pesar de ser eficiente y práctico, este enfoque tiene sus limitaciones. Una de ellas sucede si Sophia busca maximizar su suma acumulada y ganar. Con el algoritmo planteado dicho problema no se puede resolver de forma óptima y aunque nos sea costoso, podríamos usar backtracking en este caso.

¹Excepto el caso en que las monedas inicialmente son una cantidad par y son todas iguales. No hay forma de que esto no termine en empate.